

HANDOFF — FreelanceControl (TFG)

Propósito de este documento. Permitir que una sesión nueva de Claude (sin memoria de las conversaciones previas) entienda el proyecto completo y pueda analizarlo **módulo por módulo** desde hoy hasta la entrega final. Contiene el estado real verificado, el mapa de cada módulo, cómo levantar y probar todo, y qué falta. Última actualización: **2026-05-31**.

>

Si sos una sesión nueva de Claude, leé primero: este archivo, luego ESTADO_PROYECTO.md (detalle técnico exhaustivo) y docs/informe_capitulos.md (redacción académica). No asumas nada que no esté verificado acá.

0. Resumen ejecutivo en 10 líneas

- **Qué es:** sistema web de gestión financiera y fiscal para ****monotributistas argentinos****. Clasifica gastos con ML local, proyecta ingresos con Prophet, monitorea el riesgo de recategorización del Monotributo, audita registros e importa extractos bancarios.
- **Estado:** prototipo funcional **completo**. 13/13 historias de usuario y 17/17 ítems del Product Backlog del Entregable 2 implementados y verificados.
- **Calidad:** **97 tests automatizados, todos en verde**. 49 commits en main.
- **Stack:** FastAPI + PostgreSQL + React, todo en Docker Compose.
- **Diferencial:** soberanía de datos — la clasificación corre 100% local; los servicios externos (Groq) solo reciben datos numéricos agregados.
- **Lo que falta:** ver §8. Nada bloqueante para la defensa; son mejoras y un video de respaldo (excluido por decisión del alumno).

1. Cómo levantar el proyecto (reproducible)

```
# 1. Levantar los 3 servicios (db, api, frontend)
docker compose up -d
docker compose ps                # los 3 deben estar "running"
curl http://localhost:8000/health # {"status":"ok",...}

# 2. Poblar datos base (solo la primera vez, o tras recrear el volumen)
docker compose exec api python seed_categorias_monotributo.py # categorías A-K
docker compose exec api python seed_modelo_base.py            # entrena el ML base
docker compose restart api                                    # recarga el modelo en memoria
docker compose exec api python seed_demo.py                   # usuario demo + datos
```

Servicio	URL	Notas
Frontend	http://localhost:3000	React, 13 pantallas
API	http://localhost:8000	FastAPI
Swagger	http://localhost:8000/docs	Documentación interactiva
Login demo	demo@freelancecontrol.com / demo1234	María Fernández, 5 meses de datos

Trampa conocida: el modelo ML se cachea en memoria del proceso API. Si tras levantar todo el clasificador devuelve "Otros" con confianza ~0.21 para ***todo***, falta correr seed_modelo_base.py + docker compose restart api. Verificar con GET /ml/estado → debe responder usa_modelo_base: true.

2. Arquitectura y estructura

Tres capas contenedorizadas; el frontend nunca toca la BD directamente.

```
proyecto-tfg/
├── app/                                # Backend FastAPI
│   ├── main.py                        # App, CORS, registro de routers
│   ├── database.py                   # Engine, SessionLocal, get_db()
│   ├── dependencies.py               # get_current_user() (JWT → Usuario)
│   ├── models/                       (9 tablas, ~300 LOC) # SQLAlchemy ORM
│   ├── schemas/                      (~275 LOC)         # Pydantic
│   ├── services/                     (10 archivos, ~2885 LOC) # Lógica de negocio
│   └── routers/                      (12 routers, ~1200 LOC) # Endpoints REST
├── alembic/versions/                 # 5 migraciones versionadas
├── tests/                           # 11 módulos, 97 tests (~1336 LOC)
├── frontend/src/pages/               # 13 pantallas React
├── docs/                             # Entregables (ver $7)
├── seed_*.py                        # Scripts de carga (categorías, modelo, demo, ingresos)
├── evaluar_modelo.py                 # Evaluación del clasificador (CV 5-fold)
├── ESTADO_PROYECTO.md                # Detalle técnico exhaustivo
└── HANDOFF.md                       # ESTE archivo
```

Métricas de tamaño (verificadas): Services 2885 LOC · Tests 1336 · Routers 1203 · Models 301 · Schemas 275.

3. Análisis por módulos

Cada módulo lista: qué hace, archivos clave, endpoints, tests y puntos de atención para un análisis profundo. Para analizar un módulo, abrir su router + service + test correspondientes.

M1 — Autenticación y usuarios

- **Qué:** registro, login con JWT (7 días), hash bcrypt.
- **Archivos:** routers/auth.py, services/auth.py, models/usuario.py, dependencies.py.
- **Endpoints:** POST /auth/register, POST /auth/login.
- **Tests:** test_auth.py (6).
- **Decisiones clave:** usuario_id siempre del JWT (nunca del body);

anti-enumeración en login (mismo error para email inexistente o password malo).

- **Para analizar:** revisar que ningún endpoint de otros módulos acepte usuario_id por parámetro.

M2 — Ingresos

- **Qué:** CRUD de cobros. Descripción, monto, categoría, fecha.
- **Archivos:** routers/ingresos.py, models/ingreso.py, schemas/ingreso.py.
- **Endpoints:** CRUD /ingresos/ + filtros ?categoria, limite, offset.
- **Tests:** test_ingresos.py (11).
- **Para analizar:** validación monto > 0; orden por fecha descendente.

M3 — Gastos + Clasificador ML (*) (módulo central)

- **Qué:** CRUD de gastos + clasificación automática por PLN local.
- **Archivos:** routers/gastos.py, services/ml_service.py (el más grande),

services/ia_service.py (orquesta cortocircuito + ML + fallback), models/gasto.py, models/cache_clasificacion.py, models/modelo_clasificador.py.

- **Endpoints:** POST /gastos/clasificar, CRUD /gastos/, ?solo_duplicados.

- **Tests:** test_gastos.py (10), test_ml.py (12), test_ia.py (8).

- **Cómo funciona la clasificación (3 pasos):**

1. Busca corrección previa del usuario en cache_clasificacion → si existe, devuelve confianza 1.0 (normalización NFKD + sin tildes + minúsculas).
2. Si no, invoca el modelo SVM + TF-IDF.
3. Si confianza < 0.30 → "Otros" + requiere_revision.

- **Modelos:** base (600 ejemplos, 12 categorías, usuario_id NULL) + personalizados por usuario (cuando acumula 20+ gastos). Reentrenamiento en background.

- **Para analizar:** ml_service.py tiene la lógica de entrenamiento, serialización (joblib+base64 en BD) y selección NB/SVM. El dataset base está en seed_modelo_base.py.

M4 — Facturas

- **Qué:** CRUD con máquina de estados PENDIENTE → PAGADA / VENCIDA.

- **Archivos:** routers/facturas.py, models/factura.py (enum EstadoFactura).

- **Endpoints:** CRUD + PATCH /facturas/{id}/estado.

- **Tests:** test_facturas.py (10).

- **Decisión clave:** factura PAGADA es **immutable** (HTTP 409 al editar/borrar).

M5 — Auditoría (Módulo M3 del informe)

- **Qué:** 4 detectores on-demand sobre los registros.

- **Archivos:** services/auditoria.py, routers/alertas.py, models/alerta_auditoria.py (enum TipoAlerta).

- **Endpoints:** POST /alertas/ejecutar-auditoria, GET /alertas/, PATCH .../resolver.

- **Tests:** test_auditoria.py (9).

- **Los 4 detectores:** (1) duplicados [mismo monto+cat+desc en 3 días], (2) anomalías [z-score > 2σ, mín. 5 gastos/cat], (3) facturas vencidas, (4) Monotributo impago.

- **Para analizar:** parámetros ajustables al tope de auditoria.py (ventana, umbral, mínimo de datos). Borra alertas no resueltas antes de regenerar.

M6 — Proyecciones (Prophet)

- **Qué:** proyección de ingresos a 6 meses con intervalo de confianza.

- **Archivos:** services/prophet_service.py, routers/proyecciones.py, models/proyeccion.py.

- **Endpoints:** POST /proyecciones/generar, GET /proyecciones/.

- **Tests:** test_proyecciones.py (5).

- **Decisión clave:** Prophet si ≥10 ingresos; media móvil (cold start) si menos.

Cada generación reemplaza las anteriores.

M7 — Monotributo (estado fiscal)

- **Qué:** cruza facturación real + proyección Prophet → semáforo de riesgo.

- **Archivos:** services/monotributo_service.py, routers/monotributo.py, models/categoria_monotributo.py.

- **Endpoints:** GET /monotributo/estado, /pago, PATCH /categoria, /facturacion-12-meses.

- **Tests:** test_monotributo.py (8).

- **Semáforo:** verde <70%, amarillo 70-90%, rojo >90% del límite **proyectado**.

- **OJO conceptual:** porcentaje_usado = lo ya facturado (presente); el semáforo se calcula sobre el **proyectado** (futuro). Son métricas distintas.

M8 — IA externa (Groq): resumen + recomendaciones

- **Qué:** texto narrativo sobre datos **agregados**. Nunca descripciones individuales.
- **Archivos:** services/ia_service.py (funciones resumen/recomendaciones), routers/resumen.py, routers/recomendaciones.py.
- **Endpoints:** GET /resumen/financiero, GET /recomendaciones/.
- **Tests:** dentro de test_ia.py.
- **Decisión clave:** fallback local determinístico si falta GROQ_API_KEY. El sistema nunca falla por la dependencia externa.

M9 — Importación CSV/Excel

- **Qué:** importa extractos bancarios con detección heurística local.
- **Archivos:** services/csv_service.py, routers/importar.py.
- **Endpoints:** POST /importar/preview, POST /importar/confirmar.
- **Tests:** test_importar.py (13).
- **Decisiones clave:** auto-detecta separador (, ; tab); CSV/XLSX; ≤10MB (413); preview 20 filas; import transaccional atómico e idempotente (detecta duplicados).
- **Para analizar:** docs/extractos_ejemplo/ tiene 3 CSVs reales (Galicia con ;, Santander, Brubank).

M10 — Reportes PDF

- **Qué:** reporte mensual consolidado descargable.
- **Archivos:** services/reportes_service.py, routers/reportes.py, services/formato.py.
- **Endpoints:** GET /reportes/pdf?mes&anio.
- **Tests:** test_reportes.py (5).
- **Decisión clave:** ReportLab programático, 6 secciones, formato \$AR, disclaimer.

M11 — Frontend (React)

- **Qué:** 13 pantallas. Tema oscuro.
- **Archivos:** frontend/src/pages/ (Login, Register, Dashboard, Ingresos, Gastos, Facturas, Clasificador, ImportarCSV, Auditoria, Proyecciones, Monotributo, ResumenIA, Recomendaciones).
- **Tests:** no hay tests de frontend (deuda conocida, ver §8).
- **Para analizar:** volumen montado ./frontend/src:/app/src (hot-reload).

4. Base de datos

9 tablas, gestionadas con **5 migraciones Alembic** (idempotentes, se aplican solas al arrancar el contenedor):

#	Migración	Contenido
0001	initial_schema	usuarios, ingresos, gastos, facturas, proyecciones, alertas_auditoria, cache_clasificacion + enums
0002	categoria_monotributo	tabla categorías A-K
0003	modelo_clasificador	tabla del modelo ML serializado

#	Migración	Contenido
0004	alerta_monotributo_y_drop_cache	enum MONOTRIBUTO_IMPAGO + recrea cache con usuario_id
0005	indices_temporales	índices (usuario_id, fecha) para consultas AFIP 12 meses

Tablas: usuarios, ingresos, gastos, facturas, proyecciones, alertas_auditoria, categorias_monotributo, cache_clasificacion, modelos_clasificador. Detalle de columnas en ESTADO_PROYECTO.md §3.

5. Cumplimiento del Entregable 2 (verificado)

Los 10 alcances declarados: todos implementados ■ **El límite declarado (fuera de alcance):** respetado — NO se implementó (1) facturación electrónica AFIP, (2) conciliación bancaria con cuentas reales, (3) gestión de clientes como entidad propia. *No implementarlos es cumplir.*

13/13 Historias de Usuario + 17/17 Product Backlog. Cross-check completo en ESTADO_PROYECTO.md y docs/informe_capitulos.md.

6. Métricas reales del clasificador (medidas, no estimadas)

Validación cruzada 5-fold sobre 600 ejemplos (evaluar_modelo.py):

- **Accuracy global:** 76,00% (456/600 aciertos).
- **Mejores:** Monotributo F1 0.96, Impuestos 0.91, Transporte 0.88.
- **Peores:** Marketing F1 0.58, Servicios 0.63 (categorías amplias que se solapan).
- Salida cruda: docs/metricas_clasificador.txt. Gráficos:

docs/metricas_f1_por_categoria.png, docs/metricas_matriz_confusion.png.

- Reproducir: docker compose exec api python evaluar_modelo.py.

7. Entregables en docs/

Archivo	Qué es
FreelanceControl_Manual_Completo.pdf	Manual de usuario + doc técnica (22 págs)
FreelanceControl_Informe_Capitulos.pdf	Capítulos del informe: Implementación, Pruebas, Conclusiones (con anexo de métricas)
FreelanceControl_Defensa.pptx	Deck de defensa (14 slides)
GUION_DEMO_5MIN.md	Guión cronometrado de la demo en vivo
PREGUNTAS_ANTICIPADAS.md	Banco de Q&A del jurado con respuestas
DEMO.md	Guión de demo detallado + troubleshooting
informe_capitulos.md	Fuente editable de los capítulos
metricas_clasificador.txt	Salida cruda de la evaluación del ML
extractos_ejemplo/	3 CSVs de banco para probar importación
gen_*.py, gen_slides.js	Generadores reproducibles de los PDF/PNG/PPTX

8. Qué falta — pendientes hasta la entrega

Ninguno es bloqueante para la defensa. Ordenados por prioridad sugerida:

Deuda técnica reconocida (mencionada en Conclusiones del informe)

- ☐ **Tests sobre PostgreSQL real** — hoy `conftest.py` usa SQLite. Se compensó con smoke tests E2E, pero la suite de integración debería migrarse.

- ☐ **Tests de frontend** — no existen.
- ☐ CORS de producción + secretos por entorno (hoy hay separación dev, falta prod).
- ☐ Healthcheck de la BD en docker -compose antes de levantar la API.
- ☐ Logging estructurado y rate limiting en endpoints de IA.

Mejoras funcionales (trabajo futuro, NO en alcance del Entregable 2)

- ☐ Ampliar/balancear el dataset base (subir F1 de Marketing y Servicios).
- ☐ Régimen de venta de bienes (hoy solo "servicios").
- ☐ Gestión de clientes como entidad propia.
- ☐ Notificaciones de alertas (email/push).

Tareas del alumno (no automatizables por Claude)

- ☐ **Video backup de la demo** — EXCLUIDO por decisión del alumno. No hacer.
- ☐ Revisar visualmente el `.pptx` en Keynote/PowerPoint (Claude no puede renderizar pptx en este entorno; la QA fue programática: geometría + texto).
- ☐ Decidir si versionar los `Entregable2_*.docx/.pdf` (hoy untracked en git).

9. Convenciones y trampas para una sesión nueva

- **Verificá antes de afirmar.** Este proyecto tuvo varios casos de métricas o datos inventados que no coincidían con la realidad. Siempre correr el comando `/ test` y leer la salida real antes de documentar un número.

- **Idempotencia:** `seed_demo.py` se puede correr múltiples veces sin romper nada.
- **El código se hornea en la imagen Docker** (`build: .`, sin volumen para la API).

Tras editar backend hay que `docker compose build api && docker compose up -d api` para que el contenedor tome los cambios. El frontend SÍ tiene volumen (hot-reload).

- **Tests:** `docker compose exec api python -m pytest tests/ -q`.
- **Idioma:** todo el proyecto, commits y docs están en español (argentino).
- **Commits:** terminar con `Co-Authored-By: Claude <noreply@anthropic.com>`.

Comitear/pushear solo cuando el usuario lo pida.

10. Estado git

- Branch: `main`. **49 commits**. Working tree limpio salvo los dos `Entregable2_*` (untracked, decisión pendiente del alumno).

- Último commit: `43e273e docs(S4): entregables de defensa`.
- Historia completa con `git log --oneline`.

Generado el 2026-05-31. Para el detalle técnico tabla-por-tabla y endpoint-por-endpoint, ver `ESTADO_PROYECTO.md`. Para la redacción académica, ver `docs/informe_capitulos.md`.